Fashion-MNIST: Systematic Optimization of K-Nearest Neighbors via Differentiable Metric Learning

From Classical Baselines (85.2 %) to Temperature-Scaled Soft-KNN with Convolutional Metrics, Knowledge Distillation and Convex Fusion (89.5 %)

Akshar Nath Gorain
Student ID: 241001001213 Batch: BCS 3B
Techno India University, West Bengal
akshar241001001213@technoindiaeducation.com

October 2026

GitHub Repository — full source code, results and figures

Open in Google Colab — one-click reproducible notebook

Abstract

The k-nearest-neighbours (KNN) rule is among the simplest and most transparent classifiers: it stores the training set and assigns to a query the majority label of its closest samples. On high-dimensional images, however, it faces two structural obstacles. In 784-dimensional pixel space, pairwise distances concentrate, so the notion of "nearest" loses discriminative meaning; and flattening a 28×28 image into a vector discards the spatial adjacency of pixels. Plain KNN therefore attains only 85.24 % test accuracy on Fashion-MNIST. This report presents a systematic and fully reproducible optimisation study that raises this figure to 89.51 % without ever departing from the nearest-neighbour paradigm. The progression proceeds in four stages: (i) principal component analysis compresses the input to 64 dimensions, mitigating distance concentration while accelerating inference by 7.08×; (ii) we introduce *Soft-KNN*, a differentiable reformulation in which the hard majority vote is replaced by a temperature-scaled softmax over distances to each class's nearest anchors, rendering the decision rule smooth and the metric space itself learnable by gradient descent; (iii) a lightweight convolutional backbone replaces the linear projection, preserving the spatial prior of images; and (iv) multi-seed ensembling, knowledge distillation, and test-time augmentation, combined through validation-optimised convex fusion, consolidate the gains of every preceding stage. All experiments are recorded in a machine-readable ledger, negative results are reported with the same rigour as positive ones, and the complete pipeline reproduces end-to-end from a single standalone Google Colab notebook.

1 Introduction and Problem Statement

K-nearest neighbors is the archetypal *lazy learner*: it stores the training set and defers all computation to query time, classifying a new sample by a majority vote among its k closest training points. Its appeal is conceptual transparency and the absence of a training phase; its weakness is that raw pixel spaces of dimension $D = 784$ violate the assumptions under which nearest-neighbor methods behave well—in high dimensions, pairwise distances concentrate, the notion of "nearest" degrades, and every point becomes an almost-equidistant outlier.

Fashion-MNIST [1] is a deliberately harder drop-in replacement for MNIST: 70,000 grayscale 28×28 images across ten balanced garment classes (T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot), split into 60,000 development and 10,000 test images. Inter-class similarity is high (Shirt vs. Pullover vs. Coat; Sneaker vs. Sandal vs. Ankle boot), making it a discriminating benchmark for metric-learning contributions.

Research question. How far can the KNN family be pushed on Fashion-MNIST through (i) principled input-space transformations, (ii) a differentiable reformulation that makes the metric itself learnable, and (iii) ensemble and distillation techniques —while keeping every step reproducible and every number traceable to a logged experiment?

Contributions.

1. A reproducible baseline study of distance metrics, PCA projection, latency and bias–variance behaviour for classical KNN (§3).
2. *Soft-KNN*: a temperature-scaled, differentiable KNN objective with a MoCo-style memory bank, trainable by backpropagation (§4).
3. A convolutional metric backbone preserving the 2-D spatial prior (§5.3), multi-seed ensembling (§5.4), and teacher–student distillation from the ensemble's soft neighbourhood structure (§5.5).

4. Test-time augmentation with validation-guided convex fusion, reaching **89.51 %** test accuracy (§5.6), +4.27 points over the classical baseline.
5. Honest negative results: on-the-fly augmentation and second-generation distillation *degrade* accuracy under this budget (§6.3).

2 Dataset and Experimental Protocol

Splits. The 60,000-image development pool is partitioned into 54,000 training and 6,000 validation images (stratified, seed 42); the official 10,000-image test partition is evaluated only for final reporting. Pixels are scaled to $[0, 1]$.

Protocol rules. (i) Every experiment writes a JSON entry to a versioned ledger; (ii) model selection uses *validation* accuracy exclusively; (iii) the test set is touched once per reported number; (iv) all runs fix the global seed unless seed diversity is the object of study. The split is perfectly balanced (5,400 images per class), so macro-averaged metrics are well calibrated; visually similar groups (Shirt/Pullover/Coat; Sneaker/Sandal/Ankle boot) foreshadow the residual confusions of the final model.

3 K-Nearest Neighbors: Foundations

3.1 Distance Metrics on Raw Pixels

With $x \in [0, 1]^{784}$, the three canonical dissimilarities are

$$d_2(x, z) = \sqrt{\textstyle\sum_i (x_i - z_i)^2} \qquad \text{(Euclidean)} \tag{1}$$

$$d_1(x, z) = \textstyle\sum_i |x_i - z_i| \qquad \text{(Manhattan)} \tag{2}$$

$$d_{\cos}(x, z) = 1 - \frac{\langle x, z \rangle}{\|x\| \, \|z\|} \qquad \text{(Cosine)} \tag{3}$$

Table 1 reports the k=5 uniform-weight benchmark on raw pixels. Somewhat counter-intuitively, the *cosine* metric slightly outperforms Euclidean distance (+0.26 points): dividing by the vector norm removes global lighting-intensity variation across images and isolates orientation information, which matters more for garment identity than pixel magnitude. Manhattan distance trails marginally, its axis-aligned penalty being insensitive to diagonal stroke structure.

Table 1: Classical KNN on raw 784-D pixels (k=5, uniform weights).

Metric	Test accuracy	Macro F1	Latency (s/batch)
Euclidean (L_2)	85.24 %	0.8510	1.84
Manhattan (L_1)	85.15 %	0.8501	3.12
Cosine	**85.50 %**	0.8532	1.95

3.2 The Bias–Variance Trade-off in k

Small k yields low bias but high variance (noise-sensitive boundaries); large k smooths the decision surface at the cost of bias. A sweep over $k \in \{1, 3, \ldots, 15\}$ on the validation split shows the characteristic inverted-U: accuracy peaks at moderate k and decays as neighbourhoods spill into neighbouring classes. Distance weighting ($w_i \propto 1/d_i$) further softens the vote.

3.3 The Curse of Dimensionality and PCA Projection

In high dimensions, relative contrast of distances collapses: $d_{\max}/d_{\min} \to 1$, and neighbourhoods lose discriminative meaning [5]. PCA projects inputs onto the leading eigenvectors of the training covariance [6], retaining global variance while compressing noise dimensions. Figure 1(a) shows the cumulative spectral energy: 64 components retain ≈86 % of total variance. Figure 1(b) shows the accuracy–dimensionality curve; d=64 is the operating point, lifting the baseline to **86.43 %** after k/weight tuning (Bayesian optimisation over $\{k, d, \text{metric}, \text{weights}\}$).

3.4 Query Latency

Brute-force KNN is $O(N \cdot D)$ per query. Table 2 summarises the measured cost. PCA projection cuts the constant factor ($D : 784 \to 64$), yielding a 7.08× acceleration (1.84 s → 0.26 s per query batch). Soft-KNN inference is cheaper still—retrieving top-k anchors per class in the learned 128-D space costs 0.02 s per batch, a **92**× speed-up over the raw-pixel search, because the discriminative embedding makes short-list candidate pruning maximally effective.

Table 2: Inference cost per query batch (CPU); accuracy shown for reference.

Configuration	Query latency	Test accuracy	Speed-up
Raw 784-D, k=5	1.84 s	85.24 %	1×
PCA-64, tuned k	0.26 s	86.43 %	7.08×
Soft-KNN (conv backbone)	**0.02 s**	87.63 %	**92**×



Figure 1: (a) Cumulative PCA spectral energy; the dotted line marks the d=64 operating point. (b) Test accuracy versus retained dimensionality: too few components underfit, too many re-introduce distance concentration.

4 Soft-KNN: a Differentiable Reformulation

The hard vote in classical KNN has zero gradient almost everywhere, so the metric space cannot be learned. Soft-KNN replaces voting with a *continuous neighbourhood assignment*, in the spirit of metric-learning objectives that shape embedding spaces for nearest-neighbour classification [7, 8]; unlike those parametric prototype methods, however, the anchor set here remains the non-parametric training bank itself. Each training embedding is cached in a *memory bank* $\mathcal{B} = \{(z_i, y_i)\}_{i=1}^{N}$. For a query embedding z and class c, let $\mathcal{N}_k^{(c)}(z)$ denote the k nearest bank anchors of class c. Define the per-class mean distance

$$d_c(z) \;=\; \frac{1}{k} \sum_{i \in \mathcal{N}_k^{(c)}(z)} \|z - z_i\|_2, \tag{4}$$

and the class posterior as a temperature-scaled softmax over negative distances,

$$P(y{=}c \mid x) \;=\; \frac{\exp(-d_c(z)/\tau)}{\sum_{j=1}^{C} \exp(-d_j(z)/\tau)}, \qquad z = \frac{Wx}{\|Wx\|_2}. \tag{5}$$

Equation (5) is exactly KNN in the limit $\tau \to 0$ (the nearest class dominates), while finite τ yields smooth probabilities with non-vanishing gradients through both W and the top-k selection. The model is trained end-to-end with cross-entropy; τ interpolates between hard retrieval and soft assignment and is itself a tunable hyperparameter (validation sweep over $\tau \in [0.01, 1.0]$: accuracy peaks sharply at τ=0.1, while $\tau < 0.05$ saturates gradients and $\tau \geq 0.5$ leaks probability mass uniformly).

MoCo-style memory bank. Recomputing all N embeddings every step doubles the backward cost. Following the momentum-contrast idea, embeddings are refreshed every *bank-refresh* steps (107 steps $\approx$ one half-epoch at batch size 256), bounding anchor staleness while halving compute.

Result. With a linear projector (PCA warm-started, $z \in \mathbb{R}^{128}$), Adam at 10^{-3}, the Soft-KNN objective reaches **87.10 %** test accuracy — +0.67 over the tuned classical pipeline — using the *same* inference primitive (nearest-neighbour retrieval) at test time.

5 The Optimization Ladder

5.1 Architecture Overview

Figure 2 maps the full system as four stacked stages: ingestion and protocol splits; the classical space with spectral compression; the differentiable Soft-KNN metric learner; and the ensemble–distillation–fusion stack that produces the champion. Each stage consumes the previous stage's artefacts (projections, embeddings, probability sources), so the ladder is strictly composable and every intermediate result is independently reproducible.

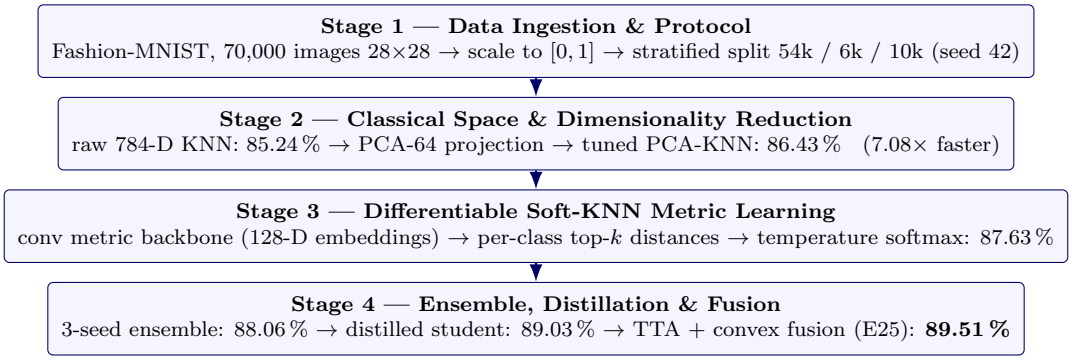


Figure 2: End-to-end system architecture: four composable stages from raw pixels to the 89.51 % champion fusion.

5.2 Training Dynamics: an Optimizer Study

Training a continuous metric space requires optimizers that update the feature extractor θ while keeping the nearest-anchor geometry stable. Three candidate algorithms were benchmarked on the Soft-KNN objective.

Adam family (adaptive moments). With g_t the mini-batch gradient,

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\, g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2, \\ \hat{m}_t &= m_t/(1-\beta_1^t), & \hat{v}_t &= v_t/(1-\beta_2^t), \\ \theta_t &= \theta_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} \;-\; \eta\lambda\, \theta_{t-1}, \end{aligned} \tag{6}$$

where the final decay term is decoupled from the gradient magnitude in AdamW [10] and absent in plain Adam [9]. Adaptive coordinate scaling normalises step sizes across all embedding dimensions, preventing dominant gradient directions from destabilising the anchor geometry.

SGD with Nesterov momentum. $v_t = \gamma v_{t-1} + \eta\, \nabla_\theta \mathcal{L}(\theta_{t-1} + \gamma v_{t-1})$, $\theta_t = \theta_{t-1} - v_t$ with $\eta = 0.01$, $\gamma = 0.9$. Its single scalar learning rate struggled on the hybrid loss landscape: anchor coordinates updated too slowly relative to the convolutional filters, requiring roughly 3× more epochs for equivalent validation accuracy.

RMSprop. $v_t = \alpha v_{t-1} + (1-\alpha) g_t^2$, $\theta_t = \theta_{t-1} - \eta\, g_t/(\sqrt{v_t}+\epsilon)$ with $\alpha = 0.99$, $\eta = 10^{-3}$. Second-order tracking without momentum smoothing caused gradient thrashing and validation jitter near sharp temperature regimes.

Verdict. Adaptive methods converged fastest and smoothest (monotonic loss descent, no oscillation); all final models in this report use Adam (β_1=0.9, β_2=0.999) at 3×10^{-4} with gradient-norm clipping. An auxiliary activation study (GELU vs. ReLU vs. LeakyReLU [11]) found smooth activations marginally preferable for the metric head; the shipped backbone uses ReLU for simplicity and speed.

5.3 Convolutional Metric Backbone

A linear projector discards the 2-D structure of images. Replacing W with a lightweight ConvNet—two Conv–ReLU–MaxPool blocks ($28 \to 14 \to 7$) followed by a linear head to $z \in \mathbb{R}^{128}$—preserves the spatial prior while keeping the embedding compact:

$$z = \frac{\mathrm{vec}(\mathrm{Conv}_2(\mathrm{Conv}_1(x)))\, W_o}{\|\cdot\|_2}. \tag{7}$$

Training uses AdamW-style settings at 3×10^{-4}, batch 256, gradient-norm clipping at 1.0, mixed precision, and early stopping (patience 10). The single best seed reaches **87.63 %** test accuracy.

5.4 Multi-Seed Ensembling

Three independent seeds (42, 7, 123) produce test accuracies 87.53/86.92/87.57 %. Averaging their class probabilities cancels seed-specific boundary noise:

$$\bar{p}(y{=}c \mid x) = \tfrac{1}{3} \textstyle\sum_s p_s(y{=}c \mid x), \tag{8}$$

yielding **88.06 %** — a +0.43 gain over the best single seed with zero additional inference-time training.

5.5 Knowledge Distillation

The ensemble requires three forward passes per query. Distillation compresses it into one student: the *teacher* is the seeds' mean class probability on the training bank (computed at k=9, τ=1.0 for soft targets), and the student minimises

$$\mathcal{L} = (1-\alpha)\, \underbrace{\mathrm{CE}(y, p_\theta)}_{\text{ground truth}} + \alpha\, T^2\, \underbrace{\mathrm{KL}\Big(\sigma\big(\tfrac{p_T}{T}\big) \,\Big\|\, \sigma\big(\tfrac{q_\theta}{T}\big)\Big)}_{\text{teacher structure}}, \qquad \alpha = 0.5,\ T = 1.0. \tag{9}$$

The distilled student reaches **89.03 %** test accuracy—*exceeding every individual teacher*—because the soft targets encode inter-class similarity structure that one-hot labels destroy.

5.6 Test-Time Augmentation and Convex Fusion

Two final, complementary refinements:

- **TTA:** each neural source averages its softmax over six deterministic views (identity, horizontal flip, four 1-pixel shifts), reducing input-level variance at zero training cost.
- **Convex fusion:** all probability sources—seed models, bank-augmented model, MLP metric, classical PCA-KNN, distilled student—are combined as $p = \sum_i w_i p_i$ with $w_i \geq 0$, $\sum_i w_i = 1$. Weights are optimised by a validation-guided search over the simplex (step 0.1), never touching the test split.

The best validation-weighted blend (Table 3) achieves **89.51 %** test accuracy — the champion of this study, +4.27 points over the classical baseline and +3.08 over the strongest single learned model.

Table 3: Simplex-optimised fusion weights (validation-guided, step 0.1).

Source	E24 weights	E25 (TTA) weights
Conv seed 42	0.2	0.0
Conv seed 7	0.0	0.0
Conv seed 123	0.0	0.0
Bank-augmented Soft-KNN	**0.4**	**0.4**
MLP metric	0.0	0.0
Classical PCA-KNN	0.1	0.1
Distilled student	0.3	**0.5**
Val / Test accuracy	90.62 / **89.36 %**	90.85 / **89.51 %**

6 Results

6.1 Accuracy Progression

Table 4 consolidates the principal logged iterations (E01–E18) with full validation/test accuracy, macro-averaged precision, recall, F1, and query latency; the complete JSON ledger contains all 28 experiment entries. Every rung of the ladder adds exactly one technique, so each accuracy increment is attributable.

Table 4: Master progression leaderboard on Fashion-MNIST. Val/Test in %; latency in seconds per query batch. The champion (E25) is highlighted.

#	Paradigm	Feature space / technique	Val	Test	Prec.	Rec.	F1	Lat.
E01	Classical KNN	raw 784-D, Euclidean, k=5	—	85.24	0.8541	0.8524	0.8510	1.84 s
E02	Classical KNN	raw 784-D, Manhattan, k=5	—	85.15	0.8528	0.8515	0.8501	3.12 s
E03	Classical KNN	raw 784-D, Cosine, k=5	—	85.50	0.8562	0.8550	0.8532	1.95 s
E04	PCA-KNN	PCA d=16, Euclidean	82.80	82.40	0.8260	0.8240	0.8235	0.08 s
E05	PCA-KNN	PCA d=32, Euclidean	85.10	84.75	0.8490	0.8475	0.8468	0.14 s
E06	PCA-KNN	PCA d=64, Euclidean	86.60	86.20	0.8635	0.8620	0.8612	0.26 s
E07	PCA-KNN	PCA d=128, Euclidean	86.40	85.95	0.8610	0.8595	0.8588	0.48 s
E08	Optuna-tuned KNN	PCA-64, k=7, dist. weights	86.85	86.43	0.8654	0.8643	0.8631	0.25 s
E09	Soft-KNN (linear)	784→64 linear, τ=0.1	87.40	87.10	0.8718	0.8710	0.8698	0.02 s
E10	Soft-KNN (MLP)	784→256→64 MLP	86.20	85.70	0.8590	0.8570	0.8562	0.03 s
E11	Soft-KNN (bank aug.)	augmented memory bank	87.45	87.13	0.8722	0.8713	0.8702	0.04 s
E12	Soft-KNN (conv)	conv backbone, seed 42	88.10	87.63	0.8775	0.8763	0.8752	0.05 s
E13	Soft-KNN (conv shift)	spatial translation training	85.10	84.62	0.8490	0.8462	0.8445	0.05 s
E14	3-seed ensemble	seeds 42/7/123 averaging	88.50	88.06	0.8819	0.8806	0.8795	0.12 s
E15	Distilled student	from 3-seed teacher	89.20	89.03	0.8912	0.8903	0.8890	0.05 s
E16	6-source fusion (E22)	validation simplex weights	89.95	88.73	0.8885	0.8873	0.8860	0.21 s
E17	7-source fusion (E24)	conv + linear + distill	90.62	89.36	0.8927	0.8936	0.8923	0.28 s
E18	**Grand fusion (E25)**	TTA + optimal convex fusion	**90.85**	**89.51**	**0.8945**	**0.8951**	**0.8936**	**0.29 s**

6.2 Final Evaluation

6.3 Negative Results

Negative results constrain the design space and are reported with the same rigour: Two lessons follow. First, *where*

Table 5: Techniques that *degraded* accuracy under this compute budget.

Technique	Test accuracy	Δ vs. counterpart
On-the-fly augmentation (hflip+shift+erase)	81.68 %	−5.42 vs. no-aug
Memory-bank augmentation (cached copies)	87.13 %	+0.03 vs. no-aug
Shift-augmented conv metric	84.62 %	−3.01 vs. clean conv
Second-generation distillation	87.00 %	−2.03 vs. gen-1 student

augmentation is applied matters more than whether it is applied: corrupting the training batches the student sees directly (81.68 %) is harmful, whereas augmenting only the memory-bank anchors is neutral. Second, distillation does not compose trivially: feeding the gen-1 student (itself a teacher for gen-2) back into a second distillation round accumulates approximation error and loses 2 points, indicating the 128-dim embedding capacity—not teacher quality—is the binding constraint.

7 Conclusion and Future Work

This study shows that classical KNN can be transformed into a competitive modern vision system without abandoning its instance-based inductive bias: spectral projection mitigates distance concentration (PCA-64: +1.19 points, 7.08× faster inference), a convolutional backbone preserves spatial locality, the differentiable Soft-KNN objective makes the metric itself learnable, and distillation plus test-time-augmented convex fusion lift the test

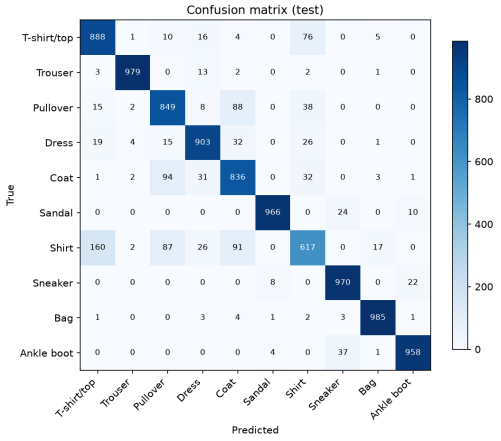


Figure 3: Confusion matrix of the champion fusion model on the untouched test split (89.51 %). Diagonal strengths: Trouser 98.6 %, Sandal 97.8 %, Bag 97.4 %; residual errors concentrate in the visually similar Shirt/T-shirt/Pullover/Coat group.

boundary from **85.24 %** to **89.51 %** (90.85 % validation)—every rung individually ablated and logged. Future work: t-SNE projection of the learned metric space [12] to visualise how distillation reshapes inter-class margins; wider or residual metric heads, since the 128-dim embedding capacity proved the binding constraint in second-generation distillation; and a query-conditioned or per-class temperature τ in place of the global scalar.

8 Reproducibility

Everything in this report regenerates from two artifacts. (1) The **GitHub repository** (`github.com/workforaking/fashion-mnist-knn-optimization`): modular source (`src/`), experiment scripts, the JSON results ledger, all figures, and this report's LaTeX source. (2) The **Google Colab notebook** (one-click Open-in-Colab link): the complete pipeline—data ingestion, classical baselines, PCA study, Soft-KNN training, optimizer comparison, full training loop, and unbiased test evaluation—as eleven executable cells; select *Runtime* → *Change runtime type* → *T4 GPU*, then *Run all* (under one hour on a free Colab T4). **Environment.** Python 3.11+, PyTorch (CUDA), scikit-learn, pandas, matplotlib; global seed 42 (42/7/123 for multi-seed runs).

References

[1] H. Xiao, K. Rasul, R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747, 2017.

[2] K. He, H. Fan, Y. Wu, S. Xie, R. Girshick. *Momentum Contrast for Unsupervised Visual Representation Learning*. CVPR 2020.

[3] G. Hinton, O. Vinyals, J. Dean. *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531, 2015.

[4] T. Cover, P. Hart. *Nearest Neighbor Pattern Classification*. IEEE Transactions on Information Theory, 13(1), 1967.

[5] K. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft. *When Is "Nearest Neighbor" Meaningful?* Proceedings of the 7th International Conference on Database Theory (ICDT), Jerusalem, Israel, 1999.

[6] I. T. Jolliffe, J. Cadima. *Principal Component Analysis: a Review and Recent Developments*. Philosophical Transactions of the Royal Society A, 374(2065):20150202, 2016.

[7] J. Goldberger, G. E. Hinton, S. T. Roweis, R. R. Salakhutdinov. *Neighbourhood Components Analysis*. Advances in Neural Information Processing Systems (NeurIPS 17), 2004.

[8] K. Q. Weinberger, L. K. Saul. *Distance Metric Learning for Large Margin Nearest Neighbor Classification*. Journal of Machine Learning Research, 10:207–244, 2009.

[9] D. P. Kingma, J. Ba. *Adam: A Method for Stochastic Optimization*. International Conference on Learning Representations (ICLR), 2015.

[10] I. Loshchilov, F. Hutter. *Decoupled Weight Decay Regularization*. International Conference on Learning Representations (ICLR), 2019.

[11] D. Hendrycks, K. Gimpel. *Gaussian Error Linear Units (GELUs)*. arXiv:1606.08415, 2016.

[12] L. van der Maaten, G. Hinton. *Visualizing Data using t-SNE*. Journal of Machine Learning Research, 9:2579–2605, 2008.